

Batch – T5**Assignment No. – 6****Title – Greedy Method****Student Name – Sharaneshwar Bharat Punjal****Student PRN – 23520011**

To apply Greedy method to solve problems of

1) **Job sequencing with Deadlines**

A) Generate table of feasible, processing sequencing, profit .

B) What is the solution generated by the function JS when $n = 7$, $(p_1, p_2, \dots, p_7) = (3, 5, 20, 18, 1, 6, 30)$, and $(d_1, d_2, d_3, \dots, d_7) = (1, 3, 4, 3, 2, 1, 2)$?

C) Input: Five Jobs with following deadlines and profits

JobID	Deadline	Profit
a	2	90
b	1	19
c	2	27
d	1	25
e	3	15

Output: Following is maximum profit sequence of jobs: c, a, e

D) Study and implement Disjoint set algorithm to reduce time complexity of JS from $O(n^2)$ to nearly $O(n)$

Algorithm:

1. Sort The jobs by decreasing order of profit
2. For each job find the largest day which can be allotted before its deadline. If found do the job and update the deadline otherwise continue. [For updating deadline we merge the Day and Day-1 together in DSU]
3. Maintain the answer during operations.

Code:

```
import java.util.*;

class Job {
    int id, deadline, profit;
    public Job(int id, int deadline, int profit) {
        this.id = id;
        this.deadline = deadline;
        this.profit = profit;
    }
}

public class Q1 {

    static boolean compareJobs(Job j1, Job j2) {
        return j1.profit > j2.profit;
    }
}
```

```
}

static List<Integer> jobSequencing(List<Job> jobs, int n) {
    jobs.sort((j1, j2) -> j2.profit - j1.profit);

    int maxDeadline = 0;
    for (int i = 0; i < n; i++) {
        maxDeadline = Math.max(maxDeadline, jobs.get(i).deadline);
    }

    int[] slots = new int[maxDeadline];
    Arrays.fill(slots, -1);

    List<Integer> sequence = new ArrayList<>();

    for (int i = 0; i < n; i++) {
        for (int j = jobs.get(i).deadline - 1; j >= 0; j--) {
            if (slots[j] == -1) { // Slot available
                slots[j] = jobs.get(i).id;
                sequence.add(jobs.get(i).id); // Add job to sequence
                break;
            }
        }
    }

    return sequence;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter the number of jobs: ");
    int n = sc.nextInt();
    List<Job> jobs = new ArrayList<>();
    System.out.println("Enter job details (id, deadline, profit):");
    for (int i = 0; i < n; i++) {
        int id = sc.nextInt();
        int deadline = sc.nextInt();
        int profit = sc.nextInt();
        jobs.add(new Job(id, deadline, profit));
    }
    List<Integer> sequence = jobSequencing(jobs, n);
    System.out.println("Maximum profit job sequence: " + sequence);
    sc.close();
}
}
```

Output:

```
Enter the number of jobs: 4
Enter job details (id, deadline, profit):
1 4 20
2 1 10
```

3 1 40

4 1 30

Maximum profit job sequence: [3, 1]

Time Complexity - $O(N)$

Space Complexity - $O(N)$

2) To implement Fractional Knapsack problem 3 objects ($n=3$).

$(w_1, w_2, w_3) = (19, 15, 10)$

$(p_1, p_2, p_3) = (25, 24, 15)$

$M = 20$

With strategy

a) Largest-profit strategy

Algorithm:

Step 1: Sort items by profit:

$p_1 = 25, p_2 = 24, p_3 = 15$.

Sorted order: Item 1, Item 2, Item 3.

Step 2: Add items to the knapsack -

Item 1: Weight = 19, Profit = 25. It fits in the knapsack.

Remaining capacity = 1, Total profit = 25.

Item 2: Weight = 15, Profit = 24. Take a fraction $(1/15)$ since only 1 unit of space is left.

Fractional profit = 1.6.

Total profit = $25 + 1.6 = 26.6$.

Result:

Chosen: Entire Item 1, fraction of Item 2.

Total Profit = 26.6.

b) Smallest-weight strategy

Algorithm:

Step 1: Sort items by weight:

$w_1 = 19, w_2 = 15, w_3 = 10$.

Sorted order: Item 3, Item 2, Item 1.

Step 2: Add items to the knapsack:

Item 3: Weight = 10, Profit = 15. It fits in the knapsack.

Remaining capacity = 10, Total profit = 15.

Item 2: Weight = 15, Profit = 24. Take a fraction $(10/15)$ since only 10 units of space are left.

Fractional profit = $24 \times (10/15) = 16$.

Total profit = $15 + 16 = 31$.

Result:

Chosen: Entire Item 3, fraction of Item 2.

Total Profit = 31.

c) Largest profit-weight ratio strategy**Algorithm:**

Step 1: Sort items by profit/weight ratio:

$p_1/w_1 = 1.32$, $p_2/w_2 = 1.6$, $p_3/w_3 = 1.5$.

Sorted order: Item 2, Item 3, Item 1.

Step 2: Add items to the knapsack:

Item 2: Weight = 15, Profit = 24. It fits in the knapsack.

Remaining capacity = 5, Total profit = 24.

Item 3: Weight = 10, Profit = 15. Take a fraction (5/10) since only 5 units of space are left.

Fractional profit = $15 \times (5/10) = 7.5$.

Total profit = $24 + 7.5 = 31.5$.

Result:

Chosen: Entire Item 2, fraction of Item 3.

Total Profit = 31.5.

Code:

```
import java.util.Arrays;
import java.util.Scanner;

public class Q2 {
    static boolean largestProfitStrategy(int a, int b, double[] profit) {
        return profit[a] > profit[b];
    }

    static boolean smallestWeightStrategy(int a, int b, double[] weight) {
        return weight[a] < weight[b];
    }

    static boolean largestProfitWeightRatioStrategy(int a, int b, double[] profit,
double[] weight) {
        return (profit[a] / weight[a]) > (profit[b] / weight[b]);
    }

    static double fractionalKnapsack(int n, double[] profit, double[] weight, int
capacity, int strategy) {
        Integer[] index = new Integer[n];
        for (int i = 0; i < n; i++) {
            index[i] = i;
        }

        if (strategy == 1) {
            Arrays.sort(index, (a, b) -> largestProfitStrategy(a, b, profit) ? -1 :
1);
        } else if (strategy == 2) {
            Arrays.sort(index, (a, b) -> smallestWeightStrategy(a, b, weight) ? -1 :
1);
        }
    }
}
```

```
    } else if (strategy == 3) {
        Arrays.sort(index, (a, b) -> largestProfitWeightRatioStrategy(a, b,
profit, weight) ? -1 : 1);
    }

    double totalProfit = 0.0;
    double currentWeight = 0.0;

    for (int i = 0; i < n; i++) {
        int idx = index[i];
        if (currentWeight + weight[idx] <= capacity) {
            totalProfit += profit[idx];
            currentWeight += weight[idx];
        } else {
            double remainingCapacity = capacity - currentWeight;
            totalProfit += (profit[idx] / weight[idx]) * remainingCapacity;
            break;
        }
    }
    return totalProfit;
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    System.out.print("Enter the number of items: ");
    int n = scanner.nextInt();

    double[] profit = new double[n];
    double[] weight = new double[n];

    System.out.println("Enter profits and weights for each item:");
    for (int i = 0; i < n; i++) {
        System.out.print("Profit for item " + (i + 1) + ": ");
        profit[i] = scanner.nextDouble();
        System.out.print("Weight for item " + (i + 1) + ": ");
        weight[i] = scanner.nextDouble();
    }

    System.out.print("Enter the capacity of the knapsack: ");
    int capacity = scanner.nextInt();

    double largestProfit = fractionalKnapsack(n, profit, weight, capacity, 1);
    System.out.println("Largest Profit Strategy: Total Profit = " +
largestProfit);

    double smallestWeight = fractionalKnapsack(n, profit, weight, capacity, 2);
    System.out.println("Smallest Weight Strategy: Total Profit = " +
smallestWeight);
}
```

```
        double largestProfitWeightRatio = fractionalKnapsack(n, profit, weight,
capacity, 3);
        System.out.println("Largest Profit Weight Ratio Strategy: Total Profit = " +
largestProfitWeightRatio);

        scanner.close();
    }
}
```

Output:

```
Enter the number of items: 3
Enter profits and weights for each item:
Profit for item 1: 25
Weight for item 1: 19
Profit for item 2: 24
Weight for item 2: 15
Profit for item 3: 15
Weight for item 3: 10
Enter the capacity of the knapsack: 20
Largest Profit Strategy: Total Profit = 26.6
Smallest Weight Strategy: Total Profit = 31.0
Largest Profit Weight Ratio Strategy: Total Profit = 31.5
```

Time Complexity - $O(N)$ **Space Complexity** - $O(N)$